Mathematical Principles of Large Language Models (LLMs)

Diogo Santos | Matěj Polák
Mathematics for Machine Learning
Instituto Superior Técnico
diogo.manuel.santos@tecnico.ulisboa.pt
matej.polak@tecnico.ulisboa.pt

Abstract

This paper presents the mathematical principles underlying Large Language Models, with emphasis on the Transformer architecture and the self-attention mechanism. We describe scaled dot-product attention, multi-head attention, positional encoding, and the encoder-decoder structure, and explain how these ideas led to GPT and BERT. A numerical example and an interactive Python demonstration are included to illustrate how attention scores, masking, softmax normalization, and contextual embeddings are computed in practice.

1 Key Concepts of the Transformer Architecture

The work on "Attention is All You Need" (1) introduced the Transformer, a sequence transduction model that dispensed entirely with recurrence and convolutions, relying solely on attention mechanisms. This section summarizes the key concepts introduced in (1), (2) and (3), namely the self-attention mechanism, positional encoding, and the overall Transformer architecture, and discusses how these contribute to the functionality of modern Large Language Models (LLMs).

Before the Transformer, sequence modeling was dominated by Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), and Gated Recurrent Units (GRUs). These architectures process sequences *step by step*, maintaining a hidden state h_t as a function of the previous state h_{t-1} and the current input. This sequential nature introduces two critical limitations:

- **No parallelization:** each step depends on the previous one, making training on long sequences slow and memory-intensive.
- **Difficulty capturing long-range dependencies:** information had to "travel" through many time steps, often getting lost (the vanishing gradient problem)

The Transformer, introduced in (1), proposed dispensing with recurrence entirely and relying solely on attention mechanisms.

1.1 The Self-Attention Mechanism

When we read a sentence, we do not process each word in isolation. Instead, we use the surrounding words to understand the meaning of each one. Self-attention (1) gives the model a similar ability: for each word in a sequence, it looks at all the other words and decides which ones are most relevant to understanding it. Intuitively, it answers the question: *when processing a given token, how much should the model attend to every other token in the sequence?*.

To understand why this matters, consider how words are represented before attention is applied. Each word in the vocabulary is initially mapped to a fixed vector in a high-dimensional space, called an

Preprint.

embedding. However, most words carry different meanings depending on context, the same word can mean very different things in different sentences. It is precisely the job of the attention mechanism to calculate what needs to be added to each generic word embedding, as a function of its context, in order to move it towards the direction that reflects its correct meaning in that particular sentence.

1.1.1 Scaled dot-product attention mechanism

We start with the **token embeddings** where a sequence of n tokens, each represented as a vector of dimension d_{model}. We stack these into an input matrix $X \in \mathbb{R}^{n \times d_{\text{model}}}$ where each row x_i is the embedding of the i-th token in the sequence.

The self-attention mechanism works through three ingredients: **queries**, **keys**, and **values**. To build an intuition for these, consider a simple analogy (4): imagine each word in a sentence is asking a question (the query) and every other word is offering an answer (the key). The better a word's answer matches the question being asked, the more attention is paid to it, and the more of its content (the value) is passed along. More formally, for each token embedding x_i, we compute three vectors by multiplying by learned weight matrices W^Q, W^K and W^V:

$$Q = XW^Q, \quad W^Q \in \mathbb{R}^{d_{\text{model}} \times d_k} \tag{1}$$

$$K = XW^K, \quad W^K \in \mathbb{R}^{d_{\text{model}} \times d_k} \tag{2}$$

$$V = XW^V, \quad W^V \in \mathbb{R}^{d_{\text{model}} \times d_v} \tag{3}$$

giving us matrices $Q, K \in \mathbb{R}^{n \times d_k}$ and $V \in \mathbb{R}^{n \times d_v}$. Each row of Q, K and V corresponds to the query, key and value vector of one token.

The **attention score** between two tokens i and j is computed as the dot product of their query and key vectors, $s_{ij} = q_i \cdot k_j$. A high value of s_{ij} means token j is highly relevant to token i. Computing this for all pairs of tokens simultaneously, we obtain the **score matrix**:

$$S = QK^T \in \mathbb{R}^{n \times n} \tag{4}$$

where entry S_{ij} contains the attention score between token i and token j.

As the dimension d_k grows large, the dot products $q_i \cdot k_j$ tend to grow large in magnitude as well. To see why, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product: $q \cdot k = \sum_{l=1}^{d_k} q_l k_l$ has mean 0 and variance d_k, since each term $q_l k_l$ has mean 0 and variance 1, and the terms are independent. This means the standard deviation of the dot product grows as $\sqrt{d_k}$, so for large d_k the scores can become very large. Large scores push the softmax function into regions where it becomes very flat, producing gradients close to zero and making learning very slow. To counteract this, the scores are **scaled** by dividing by $\sqrt{d_k}$:

$$\tilde{S} = \frac{QK^T}{\sqrt{d_k}} \tag{5}$$

This brings the variance of the scores back to 1, regardless of the dimension d_k, this is useful to keep the later **softmax** function in a well-behaved region.

Masking (prior to softmax): For the purposes of our attention pattern, we would never want words that appear later in the input to influence words that appear earlier, since otherwise they could kind of give away the answer for what comes next. In order to prevent this, what we want is for all of these spots where later tokens influence earlier ones to somehow be forced to be zero. A common way to force them to equal zero, prior to applying softmax, is to set all of those entries to be negative infinity. That way, after applying softmax, all of those spots get turned into zero, but the columns stay normalized.

Then we can apply softmax independently to each row of $\tilde{S}$. For row i:

$$\alpha_{ij} = \frac{\exp(\tilde{S}_{ij})}{\sum_{l=1}^{n} \exp(\tilde{S}_{il})} \tag{6}$$

The weight α_{ij} represents how much attention token i pays to token j. Stacking all rows, we obtain the **attention weight matrix**:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \in \mathbb{R}^{n \times n} \tag{7}$$

The Role of Softmax in the Attention Mechanism: Softmax appears in the core formula of Equation 7. Given a vector of real numbers $z = (z_1, z_2, \ldots, z_n)$, the softmax function converts it into a probability distribution:

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{n} \exp(z_j)} \tag{8}$$

The output has two important properties:

- All values are positive: $\text{softmax}(z_i) > 0$ for all i, since $\exp(z_i) > 0$ always.
- All values sum to one: $\sum_{i=1}^{n} \text{softmax}(z_i) = 1$, since we divide by the sum of all exponentials.

These two properties make the softmax output interpretable as a set of **weights**: in the attention mechanism, the output α_{ij} represents how much attention token i pays to token j, and the weights across all j sum to one.

A subtle but important aspect of softmax is the use of the exponential function $\exp(\cdot)$. Because the exponential grows very rapidly, softmax does not simply normalize the scores it *amplifies differences* between them. A score that is slightly larger than the others will receive a disproportionately larger weight after softmax is applied. The degree to which the model concentrates its attention on a small number of tokens depends on the magnitude of the scores before softmax is applied. This is directly related to the scaling factor $\frac{1}{\sqrt{d_k}}$ discussed before.

When scores are large in magnitude, the softmax output becomes very **sharp**: almost all the weight concentrates on one or a few tokens, and the rest are effectively ignored. When scores are small and close to each other, the softmax output becomes **soft**: the weights are more evenly distributed across all tokens.

The attention weight matrix $A = \text{softmax}(\tilde{S})$ directly determines what information each token collects from the rest of the sequence. Specifically, the output for each token is computed as a weighted sum of all value vectors V, using the attention weights as coefficients. For token i: $z_i = \sum_{j=1}^{n} \alpha_{ij}\, v_j$. Tokens with a high attention weight α_{ij} contribute more to the output representation of token i. If the attention weights are sharp, token i will receive information almost exclusively from one or two tokens. If they are soft, it will receive a blend of information from many tokens.

Finally, stacking all outputs into a matrix, and combining all steps, the **complete scaled dot-product attention formula** is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \tag{9}$$

This is a single, compact operation that takes a set of queries, keys and values and returns a new set of representations, each enriched by information from the most relevant tokens in the sequence, as determined by the learned weight matrices W^Q, W^K and W^V.

1.2 Multi-Head Attention

A single attention head, as described in the previous section, computes one set of attention weights over the sequence. This means the model can only capture one type of relationship between tokens at a time. In practice, however, a word can relate to other words in many different ways simultaneously grammatically, semantically, or through co reference and a single attention head would suppress this richness by averaging over all of them.

To address this, (1) proposed **multi-head attention**: instead of performing a single attention function over the full d_{model}-dimensional space, the queries, keys and values are linearly projected h times into smaller subspaces, attention is performed in parallel on each of these projections, and the results are concatenated and projected back to the original dimension:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O \tag{10}$$

where each individual head is computed as:

$$\text{head}_i = \text{Attention}(QW_i^Q,\, KW_i^K,\, VW_i^V) \tag{11}$$

and $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ are learned projection matrices.

In the base Transformer, $h = 8$ parallel attention heads are used, with $d_k = d_v = d_{\text{model}}/h = 64$. Because each head operates on a lower-dimensional projection, the total computational cost remains comparable to that of single-head attention with full dimensionality.

1.3 Positional Encoding

Because the Transformer has no recurrence and no convolution, it has no built-in sense of word order. Two sentences with the same words in different order would look identical without positional information. The solution: positional encodings are added to the input embeddings at the bottoms of the encoder and decoder stacks. The encodings have the same dimension d_{model} as the embeddings so they can be summed directly. Using sine and cosine functions of different frequencies:

$$PE_{(pos,\,2i)} = \sin\Big(\frac{pos}{10000^{2i/d_{\text{model}}}}\Big) \tag{12}$$

$$PE_{(pos,\,2i+1)} = \cos\Big(\frac{pos}{10000^{2i/d_{\text{model}}}}\Big) \tag{13}$$

where pos is the position of the token in the sequence and i is the dimension index. Each dimension of the positional encoding corresponds to a sinusoid, with wavelengths forming a geometric progression from 2π to $10000 \cdot 2\pi$. A key property of this formulation is that for any fixed offset k, PE_{pos+k} can be expressed as a linear function of PE_{pos}, allowing the model to easily learn to attend by *relative position*.

1.4 The Encoder-Decoder Architecture

The full Transformer model follows an encoder-decoder structure.

Encoder. The encoder is composed of a stack of $N = 6$ identical layers, each containing two sub-layers: a multi-head self-attention mechanism, and a position-wise fully connected feed-forward network (FFN). Residual connections followed by layer normalization wrap each sub-layer, producing the transformation:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x)) \tag{14}$$

All sub-layers and embedding layers produce outputs of dimension $d_{\text{model}} = 512$.

Decoder. The decoder is also a stack of $N = 6$ identical layers. In addition to the two sub-layers present in each encoder layer, the decoder inserts a third sub-layer that performs multi-head attention over the encoder output. Furthermore, the self-attention sub-layer in the decoder is *masked* to prevent positions from attending to subsequent positions, preserving the auto-regressive property: predictions for position i may only depend on outputs at positions less than i.

Position-wise Feed-Forward Networks. Each encoder and decoder layer also contains a fully connected feed-forward network applied identically and independently to each position:

$$\text{FFN}(x) = \max(0,\, xW_1 + b_1)\, W_2 + b_2 \tag{15}$$

with inner dimensionality $d_{ff} = 2048$.

1.5 GPT: Generative Pre-training

(2) changed the course of natural language processing (NLP). The key finding was that a model trained on a large unsupervised next-token prediction task learns representations that transfer effectively to a wide range of downstream NLP tasks after fine-tuning.

It introduced a two-step semi-supervised training framework. First, **unsupervised pre-training** is done on a large corpus of unlabeled text. The use of Transformers allows to extract phrase/sentence-level contexual relationships, rather than just word-level information. Second, **supervised fine-tuning** to a given NLP task (e.g. classification, question answering, entailment etc.) is done on a small labeled dataset. It was shown empirically that pre-training acts a regularization scheme, enabling better generalization of the model, independently of the final down-stream NLP task. This approach outperformed the existing techniques of the time, notably RNNs and LSTMs.

Unsupervised pre-training Given a large corpus of tokens $\mathcal{U} = (u_1, \ldots, u_n)$, standard *language modeling objective* is used, i.e. maximizing the likelihood that a token u_i follows the previous tokens $u_{i-k}, \ldots, u_{i-1}$:

$$L_1(\mathcal{U}) = \sum_{i=k}^{n} \log P(u_i \mid u_{i-k}, \ldots, u_{i-1}; \Theta) \tag{16}$$

where k is the context window size and Θ model parameters, trained using SGD. The model is a multi-layer Transformer decoder, as described earlier.

Supervised fine-tuning Given a labeled dataset $\mathcal{C}$, where each instance consists of a sequence of input tokens $x_1, \ldots, x_m$ and label y, the objective to maximize is:

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y \mid x_1, \ldots, x_m) \tag{17}$$

However, the authors found that including auxiliary *language modeling objective* helped learning by improving generalization of the model, and accelerating convergence. Therefore, the final fine-tuning object has the following form:

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \alpha \cdot L_1(\mathcal{C}) \tag{18}$$

For text classification, the fine-tuning is done directly as described before. Other tasks, like question answering or entailment, have structured inputs and therefore require some modification. Rather than adding task-specific architecture, *traversal-style* approach is used: the inputs are converted into an ordered sequence that the model can natively process. This avoids making extensive changes to the architecture across tasks.

1.6 BERT: Bidirectional Pre-training on the Transformer Encoder

(3) introduced BERT (Bidirectional Encoder Representations from Transformers), which established a new state-of-the-art across a wide range of NLP benchmarks. Unlike GPT, which uses a Transformer decoder and processes text autoregressively from left to right, BERT is based on the Transformer encoder and learns **bidirectional** contextual representations. This allows each token to attend to both its left and right context simultaneously.

Similarly to GPT, BERT follows a two-stage training pattern consisting of **unsupervised pre-training** on a large corpus of unlabeled text, followed by **supervised fine-tuning** on a downstream task. However, the pre-training objective differs significantly from standard language modeling. Since a bidirectional model would trivially see the token it is trying to predict, a new objective called *Masked Language Modeling* (MLM) was introduced.

Unsupervised pre-training Given a sequence of input tokens $X = (x_1, \ldots, x_n)$, a random subset of tokens is selected and replaced by a special `[MASK]` token. The model is then trained to predict the

original masked tokens using the surrounding context. Denoting the set of masked positions by M, the MLM objective can be written as

$$L_{\text{MLM}}(X) = \sum_{i \in M} \log P(x_i \mid X_{\setminus M}; \Theta), \tag{19}$$

where $X_{\setminus M}$ denotes the corrupted input sequence and Θ are the model parameters.

In addition to MLM, BERT is trained using a *Next Sentence Prediction* (NSP) objective. Given two sentences A and B, the model predicts whether B immediately follows A in the original corpus. This objective is intended to help the model learn relationships between sentences, which is useful for tasks such as question answering and natural language inference.

The overall pre-training objective is

$$L = L_{\text{MLM}} + L_{\text{NSP}}. \tag{20}$$

Supervised fine-tuning After pre-training, the same network can be adapted to a variety of downstream tasks with only minimal architectural modifications. A special `[CLS]` token is prepended to every input sequence, and its final hidden representation is used as an aggregate representation of the entire sequence.

For a classification task with labeled dataset $\mathcal{C}$, consisting of input sequences x and labels y, a task-specific output layer is added and trained using the same approach as in GPT:

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y \mid x; \Theta). \tag{21}$$

Unlike GPT, BERT does not require task-specific input transformations for most applications. The same encoder architecture can be fine-tuned for tasks such as text classification, named entity recognition, question answering, and natural language inference by modifying only the final prediction layer.

The bidirectional nature of BERT allows it to capture richer contextual information than previous unidirectional language models. As a result, BERT achieved substantial improvements over earlier approaches and became the foundation for many subsequent Transformer-based language models.

2 Numerical Example

We will demonstrate the core of the attention mechanism on a very simple artificial example. Let's consider input matrix $X \in \mathbb{R}^{5\times 3}$:

$$X = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 10 \\ 0 & 3 & 0 \\ 9 & 0 & 0 \end{bmatrix}$$

.

Its size is given by the number of input tokens $n = 5$ and embedding dimension $d_{model} = 3$. Next, let's suppose the single-head model has learned weight matrices $W_Q, W_K, W_V \in \mathbb{R}^{3\times 4}$:

$$W_Q = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 1 \end{bmatrix}, \quad W_K = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 3 & 1 \end{bmatrix}, \quad W_V = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 2 & 0 \end{bmatrix}$$

Here, the dimensions are given by embedding dimension $d_{model} = 3$ and key, query and value dimensions $d_k = d_q = d_v = 4$. Note that the dimensions can be chosen arbitrarily, but it must hold true that $d_k = d_q$ so that the dot product QK^T is well defined. With these, we can construct the key,

query and value matrices $Q, K, V \in \mathbb{R}^{5\times 4}$ by multiplying X with the respective weight matrices W_K, W_Q, W_V:

$$Q = XW_Q = \begin{bmatrix} 10 & 0 & 0 & 10 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 20 & 10 \\ 0 & 3 & 0 & 0 \\ 9 & 0 & 0 & 9 \end{bmatrix}, K = XW_K = \begin{bmatrix} 10 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 30 & 10 \\ 0 & 3 & 3 & 0 \\ 9 & 0 & 0 & 0 \end{bmatrix}, V = XW_V = \begin{bmatrix} 10 & 0 & 0 & 20 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 20 & 0 \\ 0 & 3 & 0 & 3 \\ 9 & 0 & 0 & 18 \end{bmatrix}.$$

Now comes the key step of computing the attention: the attention scores $A \in \mathbb{R}^{5\times 5}$ are given by the dot-product of Q and K^T, scaled by the square root of the key and query dimension d_k. (The scaling avoids softmax saturation and producing near-zero gradients during backpropagation.)

$$A = QK^T \cdot \frac{1}{\sqrt{d_k}} = \begin{bmatrix} 10 & 0 & 0 & 10 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 20 & 10 \\ 0 & 3 & 0 & 0 \\ 9 & 0 & 0 & 9 \end{bmatrix} \begin{bmatrix} 10 & 0 & 0 & 0 & 9 \\ 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 30 & 3 & 0 \\ 0 & 0 & 10 & 0 & 0 \end{bmatrix} \cdot \underbrace{\frac{1}{\sqrt{4}}}_{2} = \begin{bmatrix} 50 & 0 & 50 & 0 & 45 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 350 & 30 & 0 \\ 0 & 0 & 0 & 4.5 & 0 \\ 45 & 0 & 45 & 0 & 40.5 \end{bmatrix}$$

Before applying softmax, the *autoregressive* or *causal mask* is typically applied. This prevents the model from attending to future tokens upon prediction. The mask is implemented by setting these "illegal connections" to $-\infty$. Since (row-wise) Softmax is defined as $\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$, the masked entries will have value of $\frac{e^{-\infty}}{\dots} = \frac{0}{\dots} = 0$. Applying the mask and Softmax yield the attention probability matrix $P \in \mathbb{R}^{5\times 5}$ such that $P = \text{Softmax}\left(\text{Mask}\left(A\right)\right) =$

$$= \text{Softmax}\left(\begin{bmatrix} 50 & -\infty & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty & -\infty \\ 0 & 0 & 350 & -\infty & -\infty \\ 0 & 0 & 0 & 4.5 & -\infty \\ 45 & 0 & 45 & 0 & 40.5 \end{bmatrix}\right) \approx \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0.5 & 0.5 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0.011 & 0.011 & 0.011 & 0.968 & 0 \\ 0.497 & 0 & 0.497 & 0 & 0.006 \end{bmatrix}$$

Finally, using this matrix of attention probabilities, we can compute the new contextualized embeddings $Att \in \mathbb{R}^{5\times 4}$ by multiplying P and V. Note that the size of Att is different from the original matrix X, since we had set $d_v \neq d_{model}$. In practice, $d_k = d_v = \frac{d_{model}}{\#\text{ heads}}$ is usually chosen.

$$Att(Q, K, V) = PV = \begin{bmatrix} 10.000 & 0.000 & 0.000 & 20.000 \\ 5.000 & 0.000 & 0.000 & 10.000 \\ 0.000 & 0.000 & 20.000 & 0.000 \\ 0.108 & 2.903 & 0.215 & 3.118 \\ 5.022 & 0.000 & 9.945 & 10.044 \end{bmatrix}$$

3 Python Demonstration

To further demonstrate the attention mechanism, let's consider a simple sentence: "Alice put her favorite book on the bedside table." Note that in the input sentence, "Alice" has no modifiers, "book" has two modifiers "her" and "favorite", and "table" has one modifier "bedside". Suppose that the words are tokens with a 3-dimensional embedding. The embeddings were created manually to have a clear interpretation of the embeddings: noun, modifier and preposition. Next, let's consider learned 3x3 weight matrices:

$$W_Q = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, W_K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, W_V = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The values are chosen so that the matrices represent an attention head that extracts modifiers (second dimension) related to a noun (first dimension). This makes use of the fact that adjectives (and other

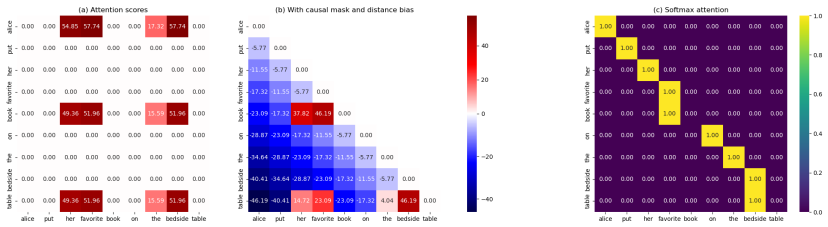

Figure 1: Attention matrices for the "description" head.

modifiers) usually precede nouns in English. Under these settings, we can produce the attention score matrix as shown in Figure 1(a).

We can immediately see two main issues with the matrix. First, attentions are computed both on preceding and following tokens. E.g., book strongly attends to preceding tokens 'her' and 'favorite' (which is correct), as well as to following 'bedside' (which is unwanted). This can be fixed by applying a causal mask.

Second, nouns attend to all previous modifiers equally. This is because no positional encoding has been applied, and therefore the attention mechanism has no way of seeing distance between tokens. For demonstration purposes, we apply a simple linear penalty to token distance:

$$[A']_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} - \beta|i-j|, \tag{22}$$

with $\beta \geq 0$ being distance-bias strength. This is of course a major simplification, since the degree of membership of a modifier to a noun is modeled by other factors than just token distance.

With these two fixes, we can see the corrected behavior of attention in Figure 1(b). The incorrect forward connections have been removed by the causal mask, and the incorrect backward connections have been mitigated by the linear penalty term.

The respective final attention weights after applying softmax can be seen in Figure 1(c). "Book" strongly attends to "favorite". The attention to "her" is weak, partly due to the distance penalty, but mostly because of smaller dot-product similarity in the key second dimension (9.5 for "her", as opposed to 10 for "favorite"). Even though "book" attends to "her" non-negligibly in (b), Softmax mostly ignores this due to strong preference of "favorite". "Table" correctly attends to "bedside". Other, non-noun words, strongly attend to themselves due to the distance penalty. Without it, attention would be equally distributed among the previous tokens (including the token itself). The matrix can now be used to get new contextual embeddings of the input tokens.

4 Additional resources

We invite the reader to explore the interactive Attention Visualizer that can be accessed at `https://2026-m4ml-2.streamlit.app`. It allows to choose different input sentences, weight matrices and settings to observe their effects on the attention scores and output embeddings. Full details of the demonstration and the source code are available at `https://github.com/polakm27/2026-m4ml-2`.

References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. & Polosukhin, I. (2017) Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pp. 5998–6008. Long Beach, CA.

[2] Radford, A., & Narasimhan, K. (2018). Improving Language Understanding by Generative Pre-Training.

[3] Devlin, J., Chang, M.W., Lee, K. & Toutanova, K. (2019) BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4171–4186. Minneapolis, MN.

[4] Sanderson, G. (2024) Visualizing Attention, a Transformer's Heart. *3Blue1Brown*. Available at: `https://www.3blue1brown.com/lessons/attention/` (Accessed: May 10, 2026).

[5] Starmer, J. (2023) Attention for Neural Networks, Clearly Explained!!! *StatQuest with Josh Starmer*. Available at: `https://www.youtube.com/watch?v=PSs6nxngL6k` (Accessed: May 10, 2026).